Software Requirements

Specification <Version 1>

<Sep 20th, 2026>

<Group 12>

<Akshaj Kapri>

<Luis Lerma>

<Luca Piu>

Prepared for

CS 250- Introduction to Software Systems

Instructor: Gus Hanna, Ph.D.

Fall 2026

Revision History

Date	Description	Author Comments
09/20/2026	Filling Sections all the way up to 3.5.6	Akshaj Kapri Luis Lerma Luca Piu <First Revision>

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature Printed Name Title Date

	<Your Name> Software Eng.
	Dr. Gus Hanna Instructor, CS 250

Table of Contents

REVISION HISTORY **1**

DOCUMENT APPROVAL **1**

1. INTRODUCTION **3**

1.1 PURPOSE **3** 1.2
SCOPE **3/4** 1.3
DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 5/6 1.4
REFERENCES **6** 1.5
OVERVIEW **6**

2. General Description 6

2.1 Product Perspective 7
2.2 Product Functions 7
2.3 User Characteristics 7
2.4 General Constraints 7
2.5 Assumptions and Dependencies 8

3. Specific Requirements 8

3.1 External Interface Requirements 8
3.1.1 User Interfaces 8
3.1.2 Hardware Interfaces 8
3.1.3 Software Interfaces 8
3.1.4 Communications Interfaces 8
3.2 Functional Requirements 9
3.2.1 Browse Movies and Showtimes 9
3.2.2 Purchase Tickets 9
3.2.3 Apply Discounts 9
3.2.4 Optional Customer Accounts and Membership 10
3.2.5 Bot Prevention 10
3.2.6 Customer Feedback 10
3.2.7 Refunds 10
3.2.8 Administrator Functions 11
3.3 Use Cases 11
3.3.1 Purchase a Ticket 12/13
3.3.2 Manage Showtimes 13
3.3.3 Return a Ticket 14
3.4 Classes / Objects 14

3.4.1 Movie 14
3.4.2 Showtime 14
3.4.3 Ticket 14
3.4.4 Order 14
3.4.5 Customer Account 15
3.4.6 Administrator 15
3.5 Non-Functional Requirements 15
3.5.1 Performance 15
3.5.2 Reliability 15
3.5.3 Availability 15
3.5.4 Security 15
3.5.5 Maintainability 15
3.5.6 Portability 16

1. Introduction

This Software Requirements Specification (SRS) provides a complete overview of the requirements for the Movie Theater Ticketing System. The system is intended to improve the process of purchasing and managing movie tickets by providing customers with access through a web browser and digital kiosks located at movie theaters. Customers will be able to use the system to select movies and showtimes, purchase tickets, and access other ticketing features. The system will also provide administrative functions that allow authorized theater personnel to manage movie showings and other ticketing information. The requirements in this document are based on information collected during client interviews and are intended to provide the development team with a clear understanding of what the system must accomplish. This SRS will serve as a reference throughout the design, implementation, and testing stages of the Movie Theater Ticketing System.

1.1 Purpose

The purpose of this Software Requirements Specification is to clearly make a Movie Theater Ticketing System that is easier and faster for us humans to book in a movie theater. The system is designed to provide customers with a convenient way to search for movie showings, select a movie and showtime, choose the number and type of tickets, select seats when applicable, complete payment, and receive their tickets. The system will also support features such as optional customer accounts, discounted tickets, digital tickets, and customer feedback. This SRS establishes a clear understanding of what the Movie Theater Ticketing System is expected to accomplish before the system moves into the design, development, and testing stages. It documents both the functional requirements, which describe the actions and services the system must provide, and the non-functional requirements, which describe qualities and constraints such as usability, security, performance, and reliability. Functional requirements describe particular behaviors of a system, while qualities such as performance and safety are treated separately as non-functional requirements.

1.2 Scope

The Movie Theater Ticketing System will include a customer ticketing interface, an administrator interface, and a central theater database. The customer portion of the system will allow customers

to interact with movie and ticket information, while the administrator portion will provide authorized theater personnel with additional controls for managing showtimes, changing theaters when necessary, and correcting purchasing errors. The system will connect to one central database
containing information for all the movies that is being provided with the data partitioned among the individual theaters.

The functions of the current version of the Movie Theater Ticketing System will not be developed as a separate mobile application because the client requested a browser-based system. The current version will not automatically locate nearby movie theaters because the client identified that capability as something that could be added in a future version of the system. Customers will also not be able to process their own refunds through the website.

The Movie Theater Ticketing System will be used to support movie ticket purchasing and ticket management for a movie theater chain in the San Diego area. The main objective of the system is to make the movie ticket purchasing process more efficient and reliable for both customers and theater employees. Customers will be able to purchase tickets up to 14 days before a scheduled showtime and until 10 minutes after the scheduled showtime begins. A customer will be limited to 20 tickets per transaction. In addition, the purchasing process will have a five-minute time limit before an unfinished transaction is abandoned. These rules provide clear limits for how and when customers can complete ticket transactions. Another objective is to support the different seating arrangements used by the theater chain. The client identified regular theaters with 150 seats and open seating and deluxe theaters with 75 seats and assigned seating. For assigned seating, the system will allow customers to view available seats and select seats as part of the ticket reservation process. This helps the system keep track of seat availability and reduces the possibility of the same available seat being selected by multiple customers.

Customer accounts will be optional rather than required, allowing customers to purchase tickets without being required to create an account. The system will support discounted ticket options, including student, seniors , children under 12 years, and military discounts. It will also support English, Spanish, and Swedish, allowing customers to interact with the system using one of the supported languages. Security and ticket control are additional goals of the application. Tickets produced by the system will be unique and designed so that they cannot be replicated, helping prevent ticket duplication and unauthorized resale. The clients require that the system should prevent automated bots from purchasing large quantities of high-demand tickets. These functions are intended to help ensure that valid tickets are issued through legitimate customer transactions.The system will also benefit theater administrators by providing administrative functions for managing the ticketing system. Authorized administrators will be able to set showtimes, make changes to theaters, and correct purchasing errors when necessary. The system will also track ticket sales and ticket prices for individual movies, providing the theater with information needed to manage its ticketing operations.

Overall, the goal of the Theater Ticketing System is to provide a centralized, accessible, and reliable ticketing application for customers and theater personnel. The system will support online and in-person ticket purchasing, enforce defined purchasing rules, manage ticket and seating availability, provide administrative controls, and maintain consistent information across

the supported theater locations.

1.3 Definitions, Acronyms, and Abbreviations

Term Definition

SRS: Software Requirements Specification; a document that describes the requirements of the software system.

Customer: A person who uses the system to view movie showtimes and purchase movie tickets.

Administrator : (Admin) An authorized theater employee who can manage showtimes, theater information, and purchasing errors.

Digital Kiosk: A self-service system located at the movie theater that customers can use to purchase tickets.

Regular Theater: A theater with 150 seats and open seating. A theater with 75

Deluxe Theater: seats and assigned seating.

Open Seating: Seating where customers do not reserve a specific seat when purchasing a ticket.

Assigned Seating: Seating where customers select and reserve a specific seat when purchasing a ticket.

Digital Ticket: An electronic movie ticket that can be provided to the customer electronically.

Showtime: The scheduled date and time that a movie begins.

UI: User Interface; the screens and controls customers and administrators use to interact with the system.

DB: Database; the stored information used by the system, such as movies, showtimes, tickets, and prices.

Term	Definition
IMDb:	Internet Movie Database; an external source that may provide movie ratings and information for the system.
Rotten Tomatoes:	A website that provides movie reviews, ratings, and information to help users learn more about movies.

1.4 References

1. Theater Ticketing Requirements: The notes that list the client's functional and non-functional requirements (ticket limits, bot protection, discounts, feedback, review integration).
2. Theater Ticketing System Questions and Answers: The customer is asked 25 questions relating to seating, discounts, budgeting, timeline, scalability, and administrator needs. 3. The flow sketches are prepared by the development team in showing the regular-ticket purchase flow, deluxe-ticket flow, discount flow, administrator flow, and exceptional conditions.
4. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE Computer Society.

1.5 Overview

The rest of this SRS describes the requirements and features of the Movie Theater System. It explains how the system will allow customers to view available movies and showtimes, select seats, purchase tickets, and receive confirmation of their purchase. It also describes how employees or administrators can manage movies, showtimes, seating, and ticket information.

The SRS is organized into sections that describe the overall system, its main functions, user requirements, system requirements, and purchase features. Each section provides more specific information about how the Movie Theater System should operate and what it needs to accomplish.

2. General Description

The Movie Theater System is designed to make it easier for customers to find movies, view available showtimes, select seats, and purchase tickets. The system will also help theater employees manage movie schedules, seating, ticket information, and customer transactions.

2.1 Product Perspective

The system is similar to existing movie theater applications and websites but is designed according to the needs provided by the client during the interviews. It may also connect with external services, such as Rotten Tomatoes, to provide additional movie information and ratings. The system will support both customers using the application and theater staff responsible for managing movie-related information.

2.2 Product Functions

The Movie Theater System will provide users with the main functions needed to search for movies and purchases tickets which will allow customers to:

- View movie ratings from Rotten Tomatoes
- Select movie and showtime
- View available movies
- Purchase movie tickets online
- Receive confirmation after purchasing tickets
- View ticket and reservation information
- view available movies and movie information

The system will also allow authorized theater staff to manage information, showtimes, and seating availability.

2.3 User Characteristics

The Movie Theater System will mainly be used by customers who want to search for movies and purchase tickets. Customers may have different levels of experience using websites and applications, so the system should be simple and easy to understand. Users should have basic
knowledge of using a computer or mobile device and accessing the internet. It may require additional access to manage and update information within the system.

2.4 General Constraints

The system will require an internet connection for users to access current movie information, showtimes, seat availability, and ticket purchasing features. It should be accessible on

commonly used computers and mobile devices. The system may depend on external services, such as Rotten Tomatoes, for movie ratings and information. If an external service is unavailable, some movie information may temporarily be unavailable. The system must also protect customers' personal and payment information when tickets are purchased. In addition, seat availability and showtimes must remain updated to prevent customers from selecting unavailable seats or incorrect showtimes.

2.5 Assumptions and Dependencies

The Movie Theater System is based on several assumptions and dependencies. It is assumed that users will have a device with a reliable internet connection and access to a supported web browser. It is also assumed that movie theaters will provide accurate and updated information about movies, showtimes, ticket prices, and available seats. If the system requires external services to become unavailable or change how their information is provided, some requirements of the Movie Theater System may need to be updated.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- Customers will interact with the Theater Ticketing System through a standard web browser. It can be accessed either from home or in person at the theater, both versions will always show the same live information. From this interface customers can search for movies , check showtimes and seat availability, choose their tickets and pay. Theater staff will have their own login protected interface where they can manage showtime, move movies to a different location, and fix customer’s orders. When the customers are checking out a countdown will be displayed so they know how much time is left before their held tickets are released and become available for other customers to purchase.

3.1.2 Hardware Interfaces

- There is no need for special hardware for this system. It just needs to run on whatever type of device the customer has. These devices include laptops , tablets , PC , and phones . The theater employees will use the same interface when helping customers buy tickets in person.

3.1.3 Software Interfaces

- The system needs to connect with the central theater database to get information about movies, showtimes, seats, and sales. It should also be connected to a payment processor for charging customers. It should also be connected to outside sources like IMDb or Rotten Tomatoes for showing rations and reviews of the movies. Even if one of the outside services goes down for a bit the rest should still be able to work normally.

3.1.4 Communications Interfaces

- Any information exchanged between a customer's device and the system, especially payment details, needs to be transmitted securely. The system also needs to keep updating ticket and seat availability in real time everywhere it's shown which will deny customers buying the same seat.

3.2 Functional Requirements

3.2.1 Browse Movies and Showtimes

- This lets a customer see what movies are playing, where it is playing and when it is playing. Along with some outside review info to help them decide.
  - ● Inputs : Which theater, and what date range they're looking at.
  - ● Processing: The system shows the list of movies and their showtimes for that theater, shows how many tickets and seats are left and displays whatever review scores or critic reviews are available.
  - ● Outputs : A browsable list of movies and showtimes with current availability. ● Error Handling : If review can't be loaded for some reason movies and it's showtime should still show up normally. A customer should be able to buy tickets even if the reviews aren't loaded.

3.2.2 Purchase Tickets

- This is the main part of the system. Buying tickets of a general seating at a regular theater or assigned seats at a deluxe one.
  - ● Inputs: Movies, showtime, number of tickets, ticket type, seat choice, and payment information.
  - ● Processing: Tickets can only be bought around 14 days before the showtime and up until 10 minutes after the showtime. No one can buy more than 20 tickets in a single order. Once someone starts checking out a five minute timer kicks in and holds their tickets or seats during that window. In a deluxe theater, the customer has to pick an actual seat, and the system can't ever let two people hold or buy the same one. Regular theaters don't have assigned seats since it's just general admission. The total price gets calculated based on what was selected and any discounts applied.
  - ● Outputs : A payment confirmation and a ticket for each of the seats purchased. ● Error Handling : If someone tries to buy more than 20 tickets the request gets rejected with an error message. If the five minute timer runs out before they finish checking out the tickets they hold will go back into availability and they're sent back to pick a showtime again. If a payment fails then the customer gets notified and can try again as long as time remains.

3.2.3 Apply Discounts

- This lets customers get a lower price on some or all of their tickets.
  - ● Inputs: The discount type for each ticket which is student, senior or military. ● Processing: Customers can mix and match the discount types across the tickets in one order

and not just apply one discount to the whole thing. While discounts are being selected the total updates automatically and customers are reminded that they may need to show ID at the theater.

● Outputs: An updated total reflecting whatever discounts were applied. ● Error handling: If someone tries to apply for more discounted tickets than they actually have in their order, that will get rejected.

3.2.4 Optional Customer Accounts and Membership

- Accounts are there if a customer wants to make one but nobody's required to make one just to buy tickets.

● Inputs: Either registration info for a new account or login info for a returning customer.

● Processing: Guests can check out whiteout ever logging in. Customers who do register get their purchase history saved. They can also buy an AMC membership card through the system and use it to reserve tickets later.

● Outputs: A working account with saved history and membership status. ● Error Handling : If someone tries to sign up with an email address that's already registered the system will stop them and give an explanation.

3.2.5 Bot Prevention

- This is meant to stop bots from getting tickets for a popular showing before real people get a chance to buy tickets.

● Inputs: Purchase attempts happening on a high demand showing.

● Processing: The system watches for purchasing patterns that look automated and blocks that activity when it is detected.

● Outputs: A message will display that lets the blocked attempt know it has been stopped.

● Error Handling: If a real customer accidentally gets blocked there should be a way to verify they're human for example an image captcha and keep going.

3.2.6 Customer Feedback

- After checkout customers will get a quick chance to say how it went.

● Inputs: A rating, picked from a row of smiley faces.

● Processing: The prompt shows up right after a purchase is complete. It is optional and doesn't hold up the ticket confirmation.

● Outputs: The rating gets saved and tied to that order.

● Error Handling: If the feedback fails to save for some reason, it shouldn't affect the purchase that already went through.

3.2.7 Refunds

- Refunds are handled by a person and not through the website.

● Inputs: A refund request that will come through an in person employee. ● Processing: The system doesn't let customers refund themselves. An employee has to process it and once they do the seat and ticket will go back into availability. ● Outputs: Confirmation of the refund and updated availability.

● Error Handling: If a refund can not be processed then the employee will be notified and it will get logged.

3.2.8 Administrator Functions

- This gives the theater staff the tools they need to actually run the system day to day. ● Inputs: Admin login and whatever changes they will be making to showtimes, theaters or orders.

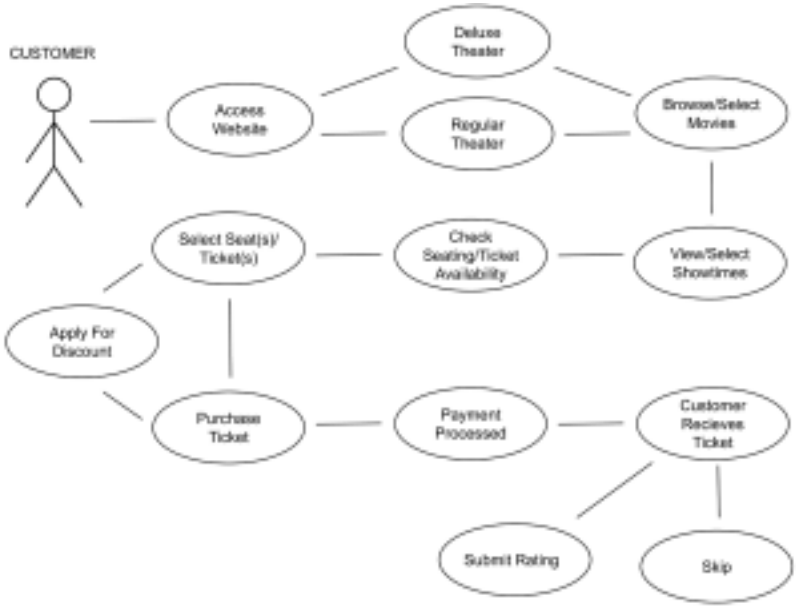
● Processing: Once logged in an administrator is able to set or change showtimes, move a movie to a different theater, fix a mistake in a customer's order, and check how many tickets were sold and at what price for any given movie.

● Outputs: Updated schedules and sales/pricing information.

● Error Handling: If an admin tries to change a showtime that already has tickets sold for it then the system asks them to confirm before actually saving the change.

3.3 Use Cases

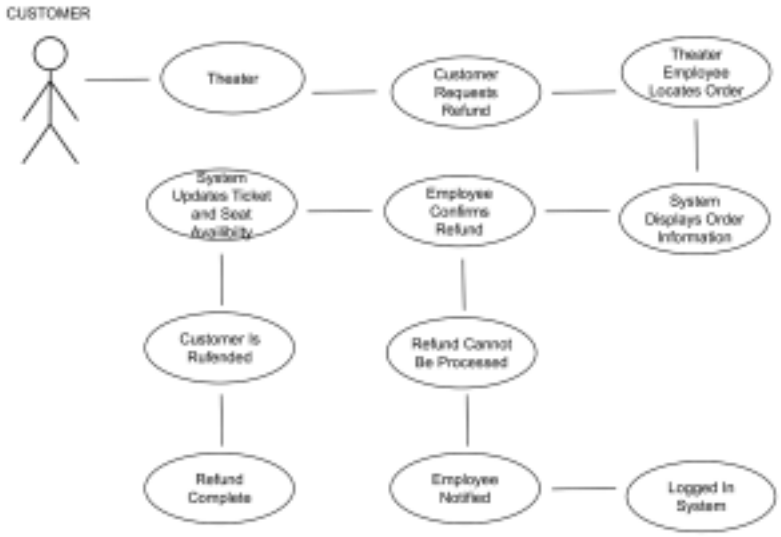
Purchase A Ticket Diagram:



Administrator Diagram:



Refund Diagram:



Name of Use Case: Purchase a Ticket

Actor(s): Customer

Entry conditions:

1. The customer selects a theater and browses the list of movies and showtimes on their website.
2. The system displays available movies along with review scores and critic quotes.
3. The customer selects a movie and showtime.
4. The system sees whether the showtime is within the allowed purchase window which is up to 14 days before the showtime and up to 10 minutes after it begins. It also displays the current seat and ticket availability.
5. The customer selects the number of tickets up to a maximum of 20 tickets per transaction.

6. The System starts a 5 minute purchase timer and holds the selected tickets. 7. If the theater is a deluxe theater, customers selected specific available seats from a seating diagram. If the theater is a regular type then this step is skipped and tickets are treated as a general admission 8. The customer will choose if they want to apply a discount (student, senior or military) 9. The system will recalculate the total based on the discount and tickets selected. 10. The Customer will enter their payment information and submit payment. 11. The system processes the payment through the payment processor.
12. The system will give a ticket for each seat purchased and display a confirmation to the customer.
13. The system prompts the customer with a row of smiley faces to rate their experience. 14. The customer submits a rating or skips the prompt.

Flow of events:

1. The customer must have access to a device that has a web browser and internet connection.
2. The selected showtime must fall within the allocated purchase limit.
3. If more than 20 tickets are requested the system will reject the request and return to step 5. If the five minute timer expires before payment is completed all the tickets held are released and the customer returns to step 3.

Name of Use Case : Manage Showtimes

Actor(s) : Administrator

Flow of events :

1. The administrator will log into the administrator interface.
2. The system verifies the administrator credentials and grants access.
3. The administrator selects a movie from the list of current or upcoming showings. 4. The administrator chooses to set a new showtime, adjust an existing showtimes, or move the movie to a different theater.
5. The system checks whether tickets have already been sold for the showtime that's being changed.
6. If the tickets have been sold the system will ask the administrator to confirm the change before proceeding.
7. The administrator confirms or cancels the change.
8. The system saves the updated showtime or theater assignment.
9. The system updates the ticket and seat availability information for all customers. **Entry conditions :**

1. The administrator must have valid credentials.

2. The administrator must be assigned access to the theater or movie being modified.

Name of Use Case : Return a Ticket

Actor(s) : Customer

Flow of events :

1. The customer requested a refund at the theater.
2. The theater employee locates the customer's order in the system.
3. The system displays the order details which includes the tickets purchased and payment method.
4. The employee confirms the refund on the customer's behalf.
5. The system releases the associated ticket and seat and places them back in availability.
6. The system processes the refund to the customer's original payment method. 7. The customer receives confirmation that the refund is complete.

Entry conditions :

1. The order being refunded must already exist in the system and not be already refunded. 2. An authorized theater employee must be available to process the refund in person.

3.4 Classes / Objects

3.4.1 Movie

- Attributes: title, rating, description, review score.
- Functions: getShowtimes(), getReviewInfo()
- Reference: 3.2.1

3.4.2 Showtime

- Attributes: movie, theater, date/time, number sold, status.
- Functions: CheckAvailability(), isWithinPurchaseWindow()
- Reference: 3.2.2

3.4.3 Ticket

- Attributes: unique ID, showtime, seat (deluxe only), discount type, price
- Functions: issueTicket(), validateTicket()
- Reference: 3.2.2, 3.2.3

3.4.4 Order

- Attributes: tickets selected, total price, timer start time, status
- Functions: startTimer(), applyDiscounts(). confirmPayment()
- Reference: 3.2.2

3.4.5 Customer Account

- Attributes: name, email, password, purchase history, membership status. ● Functions: register(), login(), viewHistory(), purchaseMembership(), useMembership() ● Reference: 3.2.4

3.4.6 Administrator

- Attributes: username, role, login credentials.
- Functions: login(), setShowtime(), correctOrder(), viewSalesReport()
- Reference: 3.2.8

3.5 Non-Functional Requirements

3.5.1 Performance

Since the client will be running 20 theaters across San Diego the system should be able to handle 1000 of people logging in at the same time without slowing down. Repetitive actions like searching for movies or making a purchase should feel quick and responsive.

3.5.2 Reliability

The system must not let two people buy or hold the same seat. If something goes wrong while making a purchase, a completed and paid order shouldn't just disappear and the system should be able to recover from the error without messing up the tickets and seating data.

3.5.3 Availability

The system needs to stay available all the time so people will be able to buy tickets anytime. Updates should happen as rarely as possible to avoid downtime. If maintenance is required it should be complete in a short period of time so that the customers barely notice

3.5.4 Security

Customers' personal and payment information should be protected during every purchase. Every type of ticket that the system provides should be a one of a kind so that it can’t be copied or reused. The system should also catch bots that are trying to bulk buy tickets for a popular movie. Admin access has to stay separate from the regular customer side and should only be given to authorized staff.

3.5.5 Maintainability

Admins should be able to change showtimes, reassign theaters, and update pricing straight from their interface without needing a developer to push in the code change. The system should also be able to log errors so that they can be tracked and fixed.

3.5.6 Portability

The system should be able to work on major web browsers and on both desktops and mobile. Customers shouldn't have to install anything to use it.

4. SOFTWARE DESIGN SPECIFICATION

Movie Theater Ticketing System

Group 12 | CS 250 - Introduction to Software Systems | Fall 2026

Team: Akshaj Kapri, Luis Lerma, Luca Piu

Design addendum: October 6, 2026 | Proposed version 2

This section is appended to the original 17-page Software Requirements Specification. The original requirements remain unchanged. Design choices and work assignments below are proposals for group review, not completed implementations.

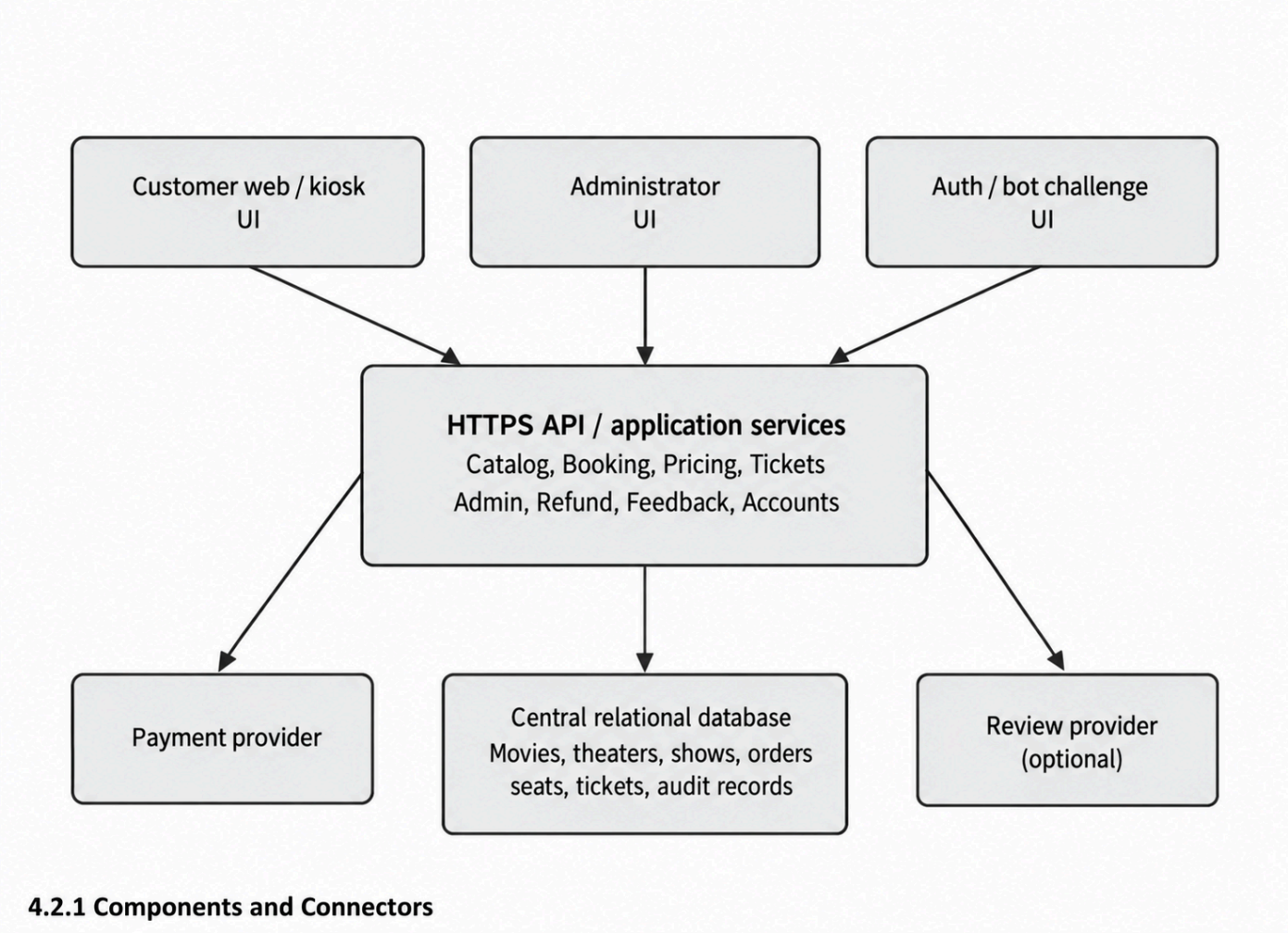
4.1 System Description

The Movie Theater Ticketing System serves a San Diego-area chain through a browser-based customer experience, in-theater kiosks, and a protected administrator interface. Customers browse movies and showtimes, buy up to 20 tickets per order, select assigned seats in deluxe auditoriums or general admission in regular auditoriums, apply eligible discounts, pay, and receive unique digital tickets. Optional accounts retain purchase history and membership information. Staff manage showtimes and orders, view sales, and perform in-person refunds. A central database separates records by theater location.

Requirements preserved: purchases open 14 days before a showtime and close 10 minutes after start; checkout holds expire after five minutes; regular auditoriums have 150 general-admission places; deluxe auditoriums have 75 assigned seats; supported languages are English, Spanish, and Swedish. Movie reviews are optional external data, and customers cannot initiate online refunds.

4.2 Software Architecture Overview

The proposed architecture uses presentation, application-service, persistence, and external-integration layers. All browser and kiosk clients use the same secure API and database-backed availability rules. Payment details are processed by a payment provider; the application stores only safe references and statuses.



4.2.1 Components and Connectors

4.2.1 Components and Connectors

Component	Purpose / interface
Customer UI / kiosk	Browser forms for browsing, seat choice, checkout, digital tickets and optional feedback. HTTPS JSON requests to the application API.
Administrator UI	Role-protected screens for schedules, order correction, sales and employee refunds. Uses the same API with

	authorization checks.

Component	Purpose / interface
Application services	Validate purchasing rules; coordinate inventory holds, pricing, payment status, ticket issuance, accounts and reports.
Central database	Stores theater-scoped movies, showtimes, seats, orders, payments, tickets, memberships and audit logs. Database transactions protect concurrent seat allocation.
Payment provider	External HTTPS payment authorization/capture/refund; service keeps provider transaction references, not raw card numbers.
Review provider	Optional movie ratings/metadata; timeouts and failures do not block ticket purchase.
Security controls	Authentication, role authorization, rate limiting, challenge for suspicious traffic, encryption in transit and audit logging.

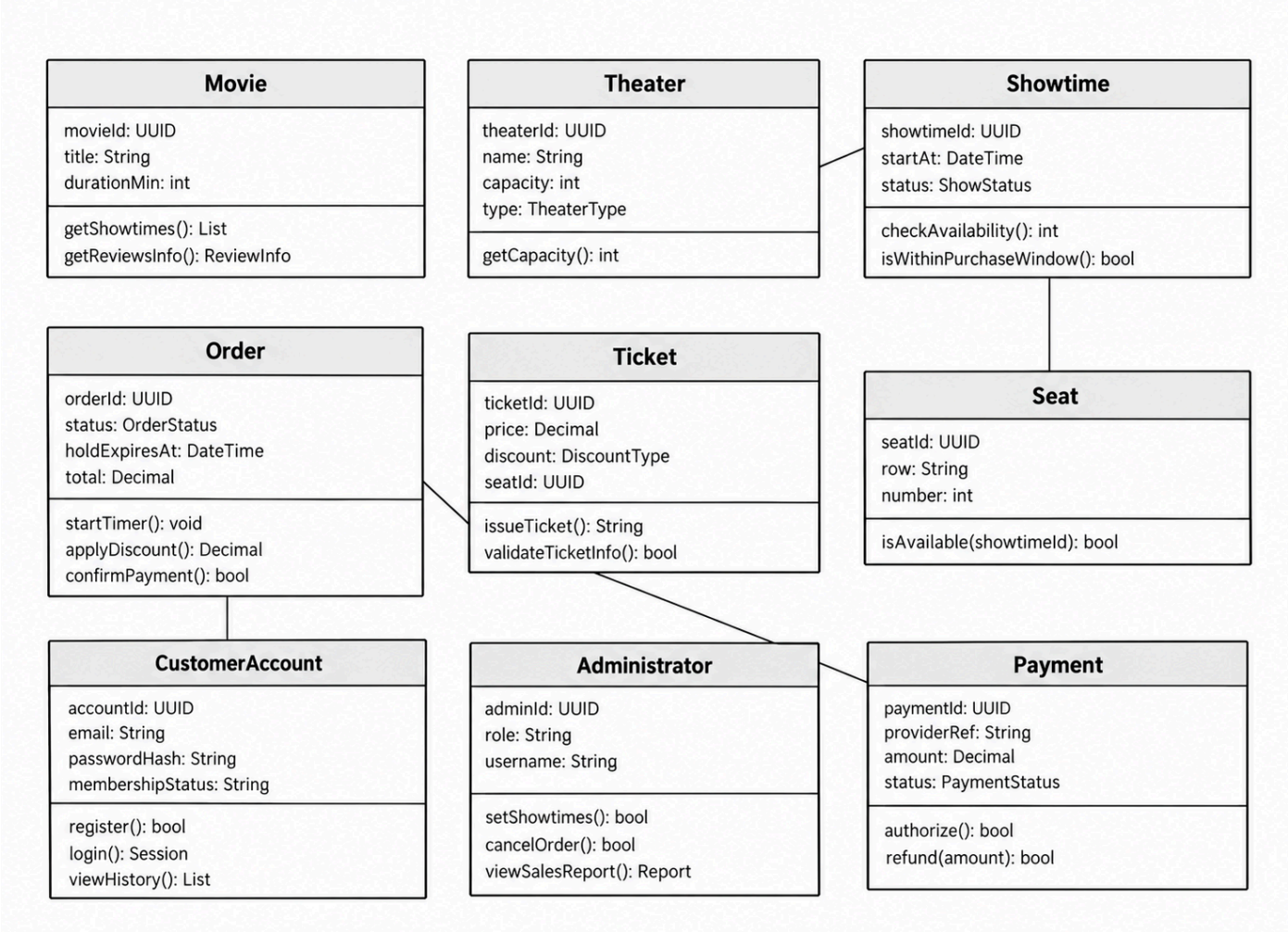
4.2.2 Key Interactions

Purchase: UI requests showtime and availability; BookingService checks the 14-day/+10-minute window and 20-ticket maximum, atomically holds capacity or seat IDs for five minutes, calculates per-ticket discounts, and requests payment authorization. Upon confirmed payment, tickets are issued with unique validation tokens and the order is committed. Failed or expired checkouts release inventory; asynchronous reconciliation prevents paid orders being lost.

Administration: authorized staff update showtimes and theater assignments, with explicit confirmation if tickets are already sold. Employee-initiated refunds invoke the payment provider and release inventory only after successful refund confirmation.

4.3 UML Class Diagram

The diagram below is a high-level developer model. The next section specifies typed fields, method signatures, and responsibilities. Associations use cardinalities; services orchestrate domain entities.



Relationships: A Movie has many Showtimes; each Showtime belongs to one Theater and can have assigned Seats for deluxe venues. An Order has 1–20 Tickets, each Ticket belongs to one Showtime and optionally one Seat. A CustomerAccount may own many Orders, but guest orders have no account. An Order has payment attempt records. Administrators manage showtimes and refunds through services; they do not own customer orders.

4.4 Class and Interface Specifications

Types: UUID = unique identifier; Decimal = currency with fixed precision; DateTime = timezone-aware timestamp; Optional<T> = nullable value; List<T> = collection; Result<T> = success/error result. All IDs and times are validated server-side.

4.4.1 Movie

Attributes: movieId: UUID; title: String; description: String; rating: String; reviewScore: Optional<Decimal>; durationMin: int.

Operations: getShowtimes(theaterId: UUID, from: DateTime, to: DateTime): List<Showtime> — retrieve matching schedules; getReviewInfo(): Optional<ReviewInfo> — fetch cached optional review metadata.

4.4.2 Theater

Attributes: theaterId: UUID; locationId: UUID; name: String; type: TheaterType

{REGULAR, DELUXE}; capacity: int. **Operations:** getCapacity(): int — return 150 for

regular or 75 for deluxe; usesAssignedSeating(): bool — true for deluxe. **4.4.3**

Showtime

Attributes: showtimeId: UUID; movieId: UUID; theaterId: UUID; startsAt: DateTime; status: ShowStatus; ticketsSold: int; basePrice: Decimal.

Operations: checkAvailability(): int — remaining unheld/unpaid capacity; isWithinPurchaseWindow(now: DateTime): bool — true from 14 days before through 10 minutes after start; updateSchedule(newTime: DateTime, newTheaterId: UUID, confirmed: bool): Result<Showtime> — enforce sold-ticket confirmation.

4.4.4 Seat

Attributes: seatId: UUID; theaterId: UUID; row: String; number: int.

Operations: isAvailable(showtimeId: UUID, now: DateTime): bool — check no live hold or sold ticket; reserve(showtimeId: UUID, orderId: UUID, expiresAt: DateTime): Result<void> — atomic unique hold.

4.4.5 Ticket

Attributes: ticketId: UUID; orderId: UUID; showtimeId: UUID; seatId: Optional<UUID>; discountType: DiscountType; price: Decimal; tokenHash: String; status: TicketStatus.

Operations: issueTicket(orderId: UUID, showtimeId: UUID, seatId: Optional<UUID>, price: Decimal): Ticket — generate unique signed token; validateTicket(token: String): bool — verify token, showtime and unused status; voidTicket(reason: String): void — invalidate on

refund.

4.4.6 Order

Attributes: orderId: UUID; customerId: Optional<UUID>; ticketIds: List<UUID>; totalPrice: Decimal; timerStart: DateTime; holdExpiresAt: DateTime; status: OrderStatus.

Operations: startTimer(now: DateTime): DateTime — set five-minute expiry; addTickets(request: List<TicketSelection>): Result<void> — enforce 1–20; applyDiscounts(types: List<DiscountType>): Decimal — compute per-ticket totals; confirmPayment(providerRef: String): Result<void> — finalize once; expire(): void — release holds.

4.4.7 CustomerAccount

Attributes: accountId: UUID; name: String; email: String; passwordHash: String; purchaseHistory: List<UUID>; membershipStatus: MembershipStatus.

Operations: register(name: String, email: String, password: String): Result<CustomerAccount> — reject duplicate email, hash password; login(email: String, password: String): Result<Session>; viewHistory(): List<Order>; purchaseMembership(paymentToken: String): Result<void>; useMembership(orderId: UUID): Result<void>.

4.4.8 Administrator

Attributes: adminId: UUID; username: String; passwordHash: String; role: AdminRole; assignedTheaterIds: List<UUID>.

Operations: login(username: String, password: String): Result<Session>; setShowtime(movieId: UUID, theaterId: UUID, startsAt: DateTime): Result<Showtime>; correctOrder(orderId: UUID, change: OrderCorrection): Result<Order>; viewSalesReport(movieId: UUID, from: DateTime, to: DateTime): SalesReport.

4.4.9 Payment

Attributes: paymentId: UUID; orderId: UUID; amount: Decimal; providerRef: Optional<String>; status: PaymentStatus.

Operations: authorize(orderId: UUID, paymentToken: String, amount: Decimal): Result<String> — send token to processor; capture(providerRef: String): Result<void>; refund(providerRef: String, amount: Decimal): Result<void> — staff-authorized only.

4.4.10 BookingService

Attributes: orderRepository: OrderRepository; inventoryRepository: InventoryRepository; paymentGateway: PaymentGateway.

Operations: holdTickets(showtimeId: UUID, selections: List<TicketSelection>, customerId: Optional<UUID>): Result<Order>; checkout(orderId: UUID, paymentToken: String): Result<List<Ticket>>; releaseExpired(now: DateTime): int — free timed-out holds.

4.4.11 DiscountPolicy

Attributes: rates: Map<DiscountType, Decimal>.

Operations: calculatePrice(basePrice: Decimal, type: DiscountType): Decimal — return configured price for ADULT, STUDENT, SENIOR, CHILD, MILITARY; validateEligibility(type: DiscountType): bool — flag ID verification when needed. Actual rates require client configuration.

4.4.12 Feedback

Attributes: feedbackId: UUID; orderId: UUID; smileRating: int; createdAt: DateTime.

Operations: submit(orderId: UUID, smileRating: int): Result<void> — optional post-purchase feedback; never blocks ticket issuance.

4.4.13 RefundService

Attributes: paymentGateway: PaymentGateway; orderRepository: OrderRepository.

Operations: refundTicket(adminId: UUID, ticketId: UUID, reason: String): Result<void> — verify staff authorization, refund original payment, void ticket and release inventory; record errors and audit trail.

4.4.14 BotProtectionService

Attributes: rateLimiter: RateLimiter; challengeProvider: ChallengeProvider.
Operations: assessPurchase(clientKey: String, showtimeId: UUID): RiskDecision — block suspicious activity; verifyChallenge(challengeToken: String): bool — permit legitimate users to continue.

4.4.15 ReviewAdapter

Attributes: reviewProvider: ReviewProvider; cache: ReviewCache.

Operations: getReview(movieId: UUID): Optional<ReviewInfo> — optional external data; return unavailable without blocking catalog.

4.5 Data Integrity, Security, and Error Handling

Use database uniqueness for (showtimeId, seatId) among active holds/issued tickets, and transactional capacity counters for general admission. Expired holds must be ignored and periodically removed. Use idempotency keys for payment callbacks and ticket issuance;

reconcile payment success after timeouts. Hash passwords; enforce administrator role and theater scope on every protected API; store only payment-provider tokens/references; log administrative changes and refund outcomes. Keep movie listings functional when review services fail. Display user-friendly errors for sold seats, expired timers, failed payment, invalid discounts and bot challenges.

4.6 Development Plan and Timeline

The following six-week schedule is a **proposed plan** for group confirmation; the SRS does not establish individual assignments or exact due dates. Work can overlap, with all members reviewing and committing changes.

Week	Tasks / deliverables Proposed lead
1	Finalize architecture, UML, database schema, repository branches and interfaces Akshaj Kapri (architecture); all review
2	Implement catalog, theater/showtime models, database migrations, admin schedule endpoints Akshaj Kapri
3	Implement checkout, seat locking, five-minute holds, discounts and order rules Luis Lerma
4	Integrate payment, unique tickets, guest/account checkout, membership and refunds Luis Lerma + Luca Piu
5	Build customer/admin UI, translations, review fallback, feedback, bot protection Luca Piu
6	Integration tests, concurrent-seat tests, security checks, documentation, deployment and demo All three members

4.6.1 Task Partition and Team Responsibilities

Member	Proposed ownership	Required GitHub evidence
Akshaj Kapri	Architecture, catalog, showtimes, database schema, diagram maintenance	Commit architecture/schema or corresponding documentation
Luis Lerma	Booking/order/ticket workflow, pricing, payment and refund integration	Commit booking/payment code or design specification

Luca Piu	Customer/admin interfaces, optional account flow, reviews, testing and usability	Commit UI/tests or corresponding documentation

Collaboration and acceptance: Each member must push at least one commit to the group repository. Use pull requests or peer review, track tasks in issues, test all acceptance rules against SRS Section 3, and submit a repository link to the updated combined PDF. Confirm actual ownership, schedule and API choices as a group before implementation.

4.7 Design Traceability

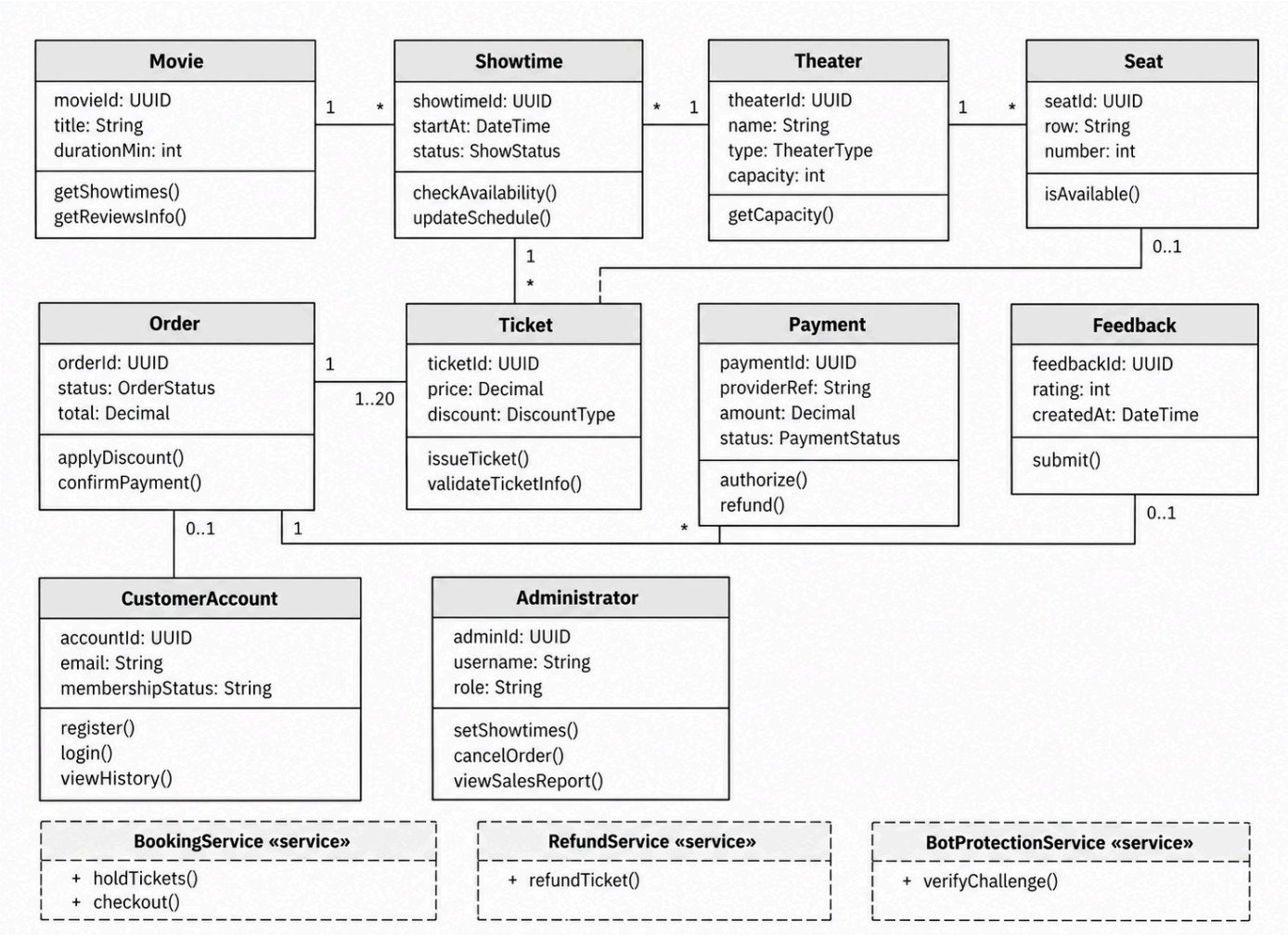
SRS requirement	Design realization
3.2.1 Browse	Movie, Showtime, Theater, ReviewAdapter, Catalog API
3.2.2 Purchase	BookingService, Order, Seat, Ticket, Payment; inventory transactions
3.2.3 Discounts	DiscountPolicy and per-ticket discount types
3.2.4 Accounts	CustomerAccount and guest order support
3.2.5 Bot prevention	BotProtectionService and challenge provider
3.2.6 Feedback	Feedback linked to completed Order
3.2.7 Refunds	RefundService restricted to authenticated employees

SRS requirement	Design realization
3.2.8 Administration	Administrator and authorized scheduling/reporting endpoints
3.5 Nonfunctional	Secure HTTPS, transaction locking, logging, responsive browser UI

Design status: Proposed design only. The original SRS is retained without editing, including any pre-existing placeholders or wording. The UML diagram displays the primary domain classes; supporting service interfaces are fully specified in Section 4.4.

4.8 Updated Design (Assignment 3)

We looked at our design again for this assignment. The overall architecture in 4.2 stays the same. What we changed is the class diagram from 4.3, because it left out some classes and operations that we had already described in 4.4. Figure 4.8 is the new version. Green means it is new in the diagram.



4.8.1 What we changed

- **Added classes.** Feedback, DiscountPolicy and ReviewAdapter, plus the services BookingService, RefundService and BotProtectionService. They were described in 4.4 but not drawn in 4.3. We need them in the diagram because we test each one.
- **Added operations** that 4.4 already listed but the diagram skipped, for example Seat.reserve(), Order.addTickets(), Order.expire(), Ticket.voidTicket(), Payment.capture() and Showtime.updateSchedule().
- **Added statuses** so we can test states. Order: NEW, HELD, PAID, EXPIRED, REFUNDED. Ticket: VALID, USED, VOID. (4.4 named the status types but not the values.)
- **Added multiplicities** from the SRS: an order has 1 to 20 tickets, a ticket has 0 or 1 seat, an order has 0 or 1 feedback, and an account can have many orders.

4.8.2 Assumptions we made for testing

The SRS leaves some numbers open, so the tests use the values below. We still need to confirm them with the group or the client.

- The purchase window includes both ends: exactly 14 days before the start and exactly 10

minutes after the start both count as allowed.
• A hold that reaches exactly 5:00 counts as expired.
• Discount percentages are not in the SRS, so we use example ones: student 10%, senior 15%, child 25%, military 20%. • Feedback is a rating from 1 to 5. More than 10 purchase attempts in 60 seconds from one client looks like a bot. • Performance target (our guess): browsing under 2 seconds and checkout under 3 seconds for 95% of requests with 1000 users.

5. TEST PLAN

Movie Theater Ticketing System | Group 12 | CS 250 - Introduction to Software Systems | Fall 2026

This test plan explains how we will check that the design in Section 4 meets the requirements in Section 3. The system is not built yet, so none of these tests has been run. The tools and schedule are proposals for the group to confirm.

5.1 Overview

We test at three levels. **Verification** asks "are we building it right?" (does the code do what the design says). **Validation** asks "are we building the right thing?" (does the system do what the customer needs). Unit and functional / integration tests are mostly verification, and system tests are validation.

Level	What it tests	How	Setup
Unit (UT-A to UT-H)	One class or method from 4.4, like Order.addTickets()	Look at the code and the rules. Test values at and next to each limit, plus normal and bad inputs.	No database or network. Payment, review service and the clock are fake.
Functional / Integration (FT-A to FT-F)	One feature from SRS 3.2 that uses several classes together, like buying a ticket	Call the API like a client would and check results, including errors and two users at once.	Real test database. Payment and review service are stubs.
System (ST-A to ST-F)	The whole product through the real screens, plus Section 3.5 (speed, security, reliability)	Walk through the use cases in 3.3, then load, break and attack it.	Test copy of the real system with a payment sandbox.

Techniques we use: boundary values (at, just below and just above each limit), equivalence classes (one example per kind of input), state tests (order, ticket, seat states), decision tables,

concurrency and load runs, fault injection (making payment or reviews fail on purpose), and the use cases from 3.3.

5.2 Test Scope Diagram

Figure 5.1 reuses our architecture from 4.2 and the classes from 4.4. Colored boxes are tested at that level. Dashed boxes are replaced by a fake or stub. Gray boxes are not tested at that level.

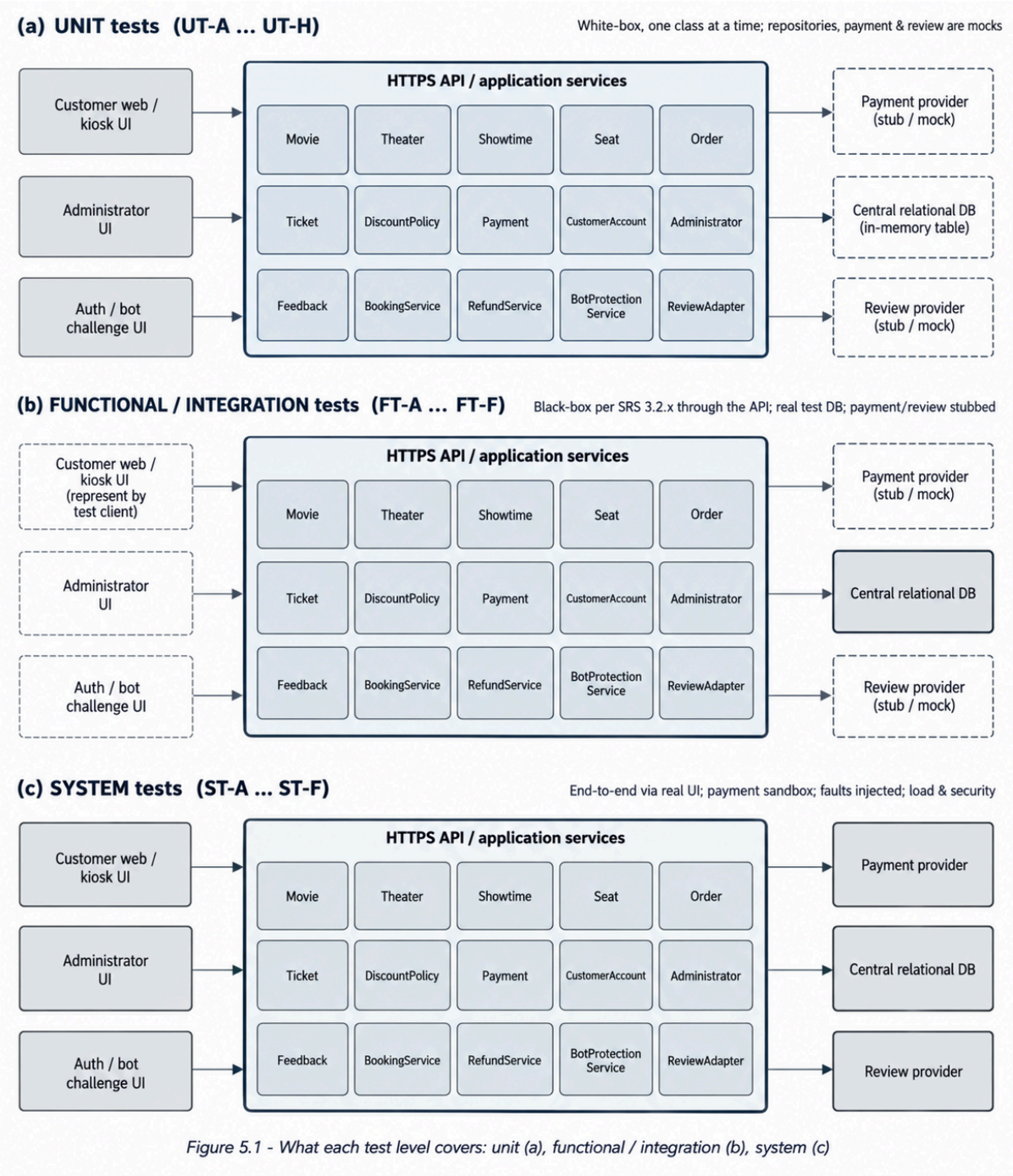


Figure 5.1 - What each test level covers: unit (a), functional / integration (b), system (c)

Cases

(appended section) Test page 2

5.3 Kinds of Failures We Cover

Failure type	Example	Caught by
Off-by-one at a limit	21 tickets accepted; sale allowed 10 min 1 s after start	Unit, functional

Timing and state	Payment accepted after the 5-minute hold ended	Unit, functional, system
Two users at once	Two people get the same seat; last place sold twice	Unit, functional, load test
Wrong calculation	Wrong discount or rounding	Unit, functional
Parts not working together	Booking and payment disagree after a failed payment	Functional, system
Security and permissions	A customer calls an admin or refund action; ticket copied	Unit, functional, system
Outside service fails	Review site is down and the movie list breaks	Unit, functional, system
Crash or lost data	Customer paid but tickets were never issued	System
Slow or does not fit device	Slow with 1000 users; broken on phone or in Spanish	System

5.4 Unit Tests

Each unit test set targets one operation from 4.4. Example data: showtime S = 2026-11-20 19:00, clock T0 = 2026-11-10 18:00:00, base ticket price $12.00.

Set	What we test	Test values	Failures it catches
UT-A	Showtime.isWithinPurchaseWindow()	now = 14 days before S minus 1 s, exactly 14 days before, 1 day before, S, S + 10 min, S + 10 min 1 s, S + 1 hour. Expected: false, true, true, true, true, false, false. Also one showtime on 2026-11-01 (clocks change).	Window edges wrong; daylight saving errors
UT-B	Order.addTickets()	Quantity -1, 0, 1, 19, 20, 21, 100. Also 15 tickets then +5 (OK) and 15 then +6 (rejected).	Limit of 20 not enforced, or checked per call instead of per order
UT-C	Order timer and payment (startTimer, confirmPayment, expire)	Pay at 4:59, 5:00 and 5:01 after start. Pay twice. Expire an order that is already paid. Declined payment then retry at 3:00 (OK) and 5:30 (rejected).	Late payment accepted; double payment; paid order released
UT-D	DiscountPolicy and Order.applyDiscounts()	Each discount type on $12.00. Mixed list [adult, adult, student, senior] = $45.00. More discounts than tickets. $9.99 with senior = $8.49.	Wrong price, rounding, too many discounts

UT-E	Seat, Showtime and Theater (reserve, isAvailable, getCapacity)	Seat free, held, held but expired, sold. Reserve the same seat twice. Regular = 150 and no seat choice; deluxe = 75 with seat choice. Counter 150 down to 0.	Double booking; expired hold still blocks; wrong capacity
UT-F	Ticket (issue, validate, void)	Make 1000 tickets and check all codes differ. Change one character of a code. Use the code of another showtime. Validate twice. Validate after void.	Duplicate or fake tickets; refunded ticket still works
UT-G	CustomerAccount and Administrator	New email, same email twice, same email in other letter case, wrong password, password stored as hash. Admin change with 0 or 3 tickets sold, confirmed or not; admin of another theater.	Duplicate accounts; plain-text password; change without confirmation
UT-H	ReviewAdapter, Feedback, BotProtection Service	Review service OK, error, bad data, timeout. Rating 0, 1, 5, 6; saving fails. 1, 10, 11, 50 attempts in 60 s; valid, wrong and reused challenge.	Review outage breaks the list; feedback blocks tickets; bots not stopped

These values come straight from the numbers in the SRS (14 days, 10 minutes, 20 tickets, 5 minutes, 150 and 75 seats), so each rule has a test that should pass at the limit and one that should fail just past it. Sample cases UT_1 to UT_4 in Section 6 are written out from UT-A to UT-D.

5.5 Functional / Integration Tests

These tests call the API like a real client, with a real test database. Each set takes one requirement from SRS 3.2 and turns its inputs, processing, outputs and error handling into test values

Set	**Feature (SRS)**	**Classes together**	**Test values and expected result**	**Failures it catches**
FT-A	Browse movies (3.2.1)	Movie, Showtime, Theater, ReviewAdapter, DB	Date ranges: empty, one day, 14 days, past. Theater with no showtimes. Review service OK, slow or down. The list always shows with places left; with reviews down it shows without scores.	Missing availability; reviews block the list

FT-B	Buy tickets, regular and deluxe (3.2.2)	BookingService, Order, Seat, Showtime, DB	Quantity 1, 20, 21. Places left 150, 20, 1, 0. Deluxe with no seat gives an error; free seat is held; taken seat gives SEAT_UNAVAILABLE. Two clients want the same seat (100 runs). Overlapping seat sets [A1,A2] and [A2,A3]: all or nothing. Hold expires, then someone else buys.	Overselling; double booking; half-finished holds
FT-C	Discounts (3.2.3)	Order, DiscountPolicy, Ticket	Mix of discount types on 1 to 20 tickets. Total updates after each change. More discounts than tickets is rejected. Ticket prices add up to the order total.	Wrong totals; discount on whole order
FT-D	Accounts, bots, feedback (3.2.4 to 3.2.6)	CustomerAccount, BotProtectionService, Feedback	Guest checkout works. Register, buy, history shows the order. Duplicate email. 50 purchases in 10 s from one client are blocked; a blocked person can solve the challenge. Rating 1 to 5, skip, or rating for an unpaid order. Failed feedback does not stop the ticket.	Guest forced to register; bots succeed; feedback blocks ticket
FT-E	Refunds (3.2.7)	RefundService, Payment, Ticket, Seat	Customer, employee and admin roles. Payment provider succeeds or fails. Already refunded. One of two tickets refunded. Only employees succeed, and the seat comes back only after the provider confirms.	Self-refund; seat freed too early; failure not logged
FT-F	Admin functions (3.2.8)	Administrator, Showtime, Theater, reports, log	Change a showtime with 0 or some tickets sold, with and without confirmation. Move a movie to another theater. Admin of another theater is rejected. Sales report equals the sum of paid tickets.	Silent change of sold showtimes; wrong totals; no log entry

The seat test (FT-B) checks the database directly, so it also catches bugs that the API answers might hide. Sample cases FT_1, FT_2 and FT_3 come from FT-B, FT-F and FT-E.

5.6 System Tests

System tests use the real screens (browser and kiosk). They check the use cases in 3.3 and the non-functional requirements in 3.5.

Set	Scenario (SRS)	What we do	Pass when
ST-A	Buy a ticket end to end (3.3.1)	Deluxe in browser and regular at kiosk; guest and account; mixed discounts; stay idle 5 minutes on the payment page; declined payment then success.	Totals are right, one unique ticket per seat, sold seats disappear for others, timeout sends user back.
ST-B	Manage showtimes and refund (3.3.2, 3.3.3)	Admin moves a showtime with sold tickets and confirms; cancels another change; employee refunds an order and the seat is sold again.	Kiosk and browser agree within 5 s; cancelled change changes nothing.
ST-C	Security (3.5.4)	Customer calls admin and refund actions; admin edits another theater; HTTP goes to HTTPS; injection text in search and login; repeated wrong logins; copy a ticket code; look in DB for card numbers.	Always refused; no card data stored; second use of a ticket rejected.
ST-D	Speed and many users (3.5.1, 3.5.2)	1000 users at once on 20 theaters (70% browse, 20% buy, 10% account pages). 50 of them fight for the last 10 seats.	Browse under 2 s and checkout under 3 s for 95%, errors under 1%, no seat sold twice.
ST-E	Recovery (3.5.2)	Kill the booking service right after payment; make the ticket step fail; send the same payment message twice.	Tickets issued within 5 minutes, exactly one payment and one set of tickets.
ST-F	Browsers, languages, outages (3.5.3, 3.5.6)	Chrome, Firefox, Safari, Edge; phone and tablet; English, Spanish, Swedish; review service down; payment timeout; maintenance.	Same behavior everywhere; purchase works without reviews; open orders survive.

ST-A and ST-B replay the use cases from 3.3, so passing them shows the flows the client approved really work. Sample cases ST_1, ST_2 and ST_3 come from ST-A, ST-D and ST-E.

5.7 Requirements and Tests

Requirement	Unit	Functional / Integration	System	Sample cases
3.2.1 Browse	UT-E, UT-H	FT-A	ST-A, ST-F	-
3.2.2 Purchase	UT-A, B, C, E, F	FT-B	ST-A, ST-D, ST-E	UT_1, UT_2, UT_3,

				FT_1, ST_1
3.2.3 Discounts	UT-D	FT-C	ST-A	UT_4, ST_1
3.2.4 to 3.2.6 Accounts, bots, feedback	UT-G, UT-H	FT-D	ST-A, ST-C	ST_1 (skip feedback)
3.2.7 Refunds	UT-F	FT-E	ST-B	FT_3
3.2.8 Administration	UT-G	FT-F	ST-B, ST-C	FT_2
3.5 Performance, reliability	UT-C, UT-E	FT-B	ST-D, ST-E	FT_1, ST_2, ST_3
3.5 Security, portability, availability	UT-F, UT-G, UT-H	FT-D, FT-E	ST-C, ST-F	-

5.8 Tools, Pass Rules and Schedule

- **Tools (proposed):** a unit test framework with mocks, an API client, browser automation, a load tool (k6 or JMeter), a payment sandbox and a seeded test database.
- **We are done when:** all P0 cases pass, at least 95% of all cases pass, there are no open critical bugs, every requirement in 5.7 has a test that ran, and the seat conflict test shows no double booking in 100 runs. Failures go in GitHub issues with the test ID.
- **Schedule (proposal, matches 4.6):** unit tests are written with each feature in Weeks 2 to 5, functional tests start in Week 4, system tests are in Week 6. Luca does the UI and system tests, Akshaj the catalog and admin tests, Luis the booking, payment and refund tests (group to confirm).

6. TEST CASE SAMPLES

Here are our 10 sample test cases, written with the course Excel template. All 10 are in the file **Group12_TestCases.xlsx**. The ID tells the level: **UT_** unit, **FT_** functional / integration, **ST_** system. P0 means it must pass before release and P1 means important. The system is not built yet, so Actual Result is "Not executed" and Status is "Not Run" for now.

6.1 Summary

TestCaseId	Level	Component	Priority	Plan set	SRS
UT_1	Unit	Showtime.isWithinPurchaseWindow()	P0	UT-A	3.2.2
UT_2	Unit	Order.addTickets()	P0	UT-B	3.2.2
UT_3	Unit	Order.startTimer() / expire() /	P0	UT-C	3.2.2, 3.3.1

		confirmPayment()			
UT_4	Unit	Order.applyDiscounts() / DiscountPolicy	P1	UT-D	3.2.3
FT_1	Funct. / Integration	BookingService.holdTickets() + Seat + database	P0	FT-B	3.2.2, 3.5.2
FT_2	Funct. / Integration	Administrator.setShowtime() / Showtime.updateSchedule()	P1	FT-F	3.2.8, 3.3.2
FT_3	Funct. / Integration	RefundService.refundTicket() + Payment + Ticket	P1	FT-E	3.2.7, 3.3.3
ST_1	System	End-to-end purchase (kiosk/browser -> API -> DB -> payment sandbox)	P0	ST-A	3.3.1, 3.2.3
ST_2	System	Whole system - performance and concurrency	P1	ST-D	3.5.1, 3.5.2
ST_3	System	Whole system - recovery after failure	P0	ST-E	3.5.2

We have 4 unit, 3 functional / integration and 3 system cases, so every level has at least two.

6.2 Three Full Examples (one per level)

Test Case Id	UT_3
Component	Order.startTimer() / expire() / confirmPayment()
Priority	P0
Description / Test Summary	Unit: verify the five-minute checkout hold. Payment is accepted before expiry; after expiry it is rejected and the held places are released (SRS 3.2.2, 3.3.1).
Pre-requisites	Order with 2 held tickets. startTimer(now = 2026-11-10 18:00:00) has been called, so holdExpiresAt = 18:05:00. PaymentGateway is a stub that always authorizes.
Test Steps	1. Check that holdExpiresAt equals 18:05:00. 2. Call confirmPayment(ref) with the clock at 18:04:59. 3. Create a second identical Order; call confirmPayment(ref) with the clock at 18:05:00. 4. Create a third identical Order; call confirmPayment(ref) with the clock at 18:05:01.

	5. For the Orders from steps 3-4 call Seat.isAvailable() / checkAvailability().
Expected Result	1. holdExpiresAt = 18:05:00. 2. Success; status = PAID (finalized once only). 3. Rejected: HOLD_EXPIRED (boundary treated as expired; to be confirmed by the group). 4. Rejected: HOLD_EXPIRED; status = EXPIRED. 5. Held places are available again; gateway.authorize() was never called for the expired Orders.
Actual Result	Not executed (implementation pending)
Status	Not Run
Test Executed By	TBD

Test Case Id	FT_1
Component	BookingService.holdTickets() + Seat + database
Priority	P0
Description / Test Summary	Functional: two customers request the same assigned seat in a deluxe theater at the same moment; exactly one may succeed (SRS 3.2.2, 3.5.2).
Pre-requisites	Test database seeded with one deluxe theater (75 seats) and one showtime inside the purchase window. Seat C7 is free. Two API clients (guest A, guest B) are ready. PaymentGateway is stubbed.
Test Steps	1. Both clients send holdTickets(showtimeId, [seat C7], guest) at the same time (release them from a common barrier). 2. Read both responses. 3. Query the database for active holds on (showtimeId, C7). 4. Repeat steps 1-3 for 100 iterations, each with a fresh seat.
Expected Result	2. Exactly one response is an Order in status HELD; the other is Error SEAT_UNAVAILABLE. 3. Exactly one active hold row exists for (showtimeId, C7). 4. 100 / 100 iterations show no double hold and no unhandled exception; the uniqueness constraint on (showtimeId, seatId) is never violated.
Actual Result	Not executed (implementation pending)
Status	Not Run
Test Executed	TBD

By	

Test Case Id	ST_1
Component	End-to-end purchase (kiosk/browser -> API -> DB -> payment sandbox)
Priority	P0
Description / Test Summary	System: a guest buys 3 tickets with mixed discounts for a deluxe showtime through the real UI and receives unique digital tickets (use case 'Purchase a Ticket', SRS 3.3.1).
Pre-requisites	Staging environment with real UI, API, database, and payment-provider sandbox. Deluxe showtime within the purchase window with seats D4, D5, D6 free. Fixture rates: STUDENT 10%, SENIOR 15%; base price $15.00. Second browser (Browser B) is open on the same showtime.
Test Steps	1. In Browser A (no login) select the deluxe theater, the movie, and the showtime. 2. Choose 3 tickets, select seats D4, D5, D6. 3. Set discounts: ticket 1 ADULT, ticket 2 STUDENT, ticket 3 SENIOR. Note the displayed total and ID reminder. 4. Observe the countdown, then pay with the sandbox test card. 5. Note the confirmation and the three tickets; skip the smiley feedback prompt. 6. In Browser B reload the seat map. 7. Compare the three ticket tokens.
Expected Result	3. Total updates automatically to $41.25 (15.00 + 13.50 + 12.75); reminder to bring ID is shown. 4. A 5-minute countdown is visible; payment is accepted. 5. Confirmation shows 3 tickets, one per seat; the skipped feedback prompt does not block anything. 6. Seats D4-D6 appear as sold and cannot be selected. 7. Three different tokens; each passes validateTicket once. Only a provider reference (no card number) is stored.
Actual Result	Not executed (implementation pending)
Status	Not Run
Test Executed By	TBD

6.3 Test Cases Link :

https://github.com/Akushaj/Software-Design-Specification/blob/main/docs/Group12_Test_Cases.xlsx